

Institute of Business Administration, Karachi
Department of Computer Science
Spring 2026

Parallel Soft Clustering for
Clinical Notes with OpenMPI
Project 21 — Milestone 2 Submission Report
MPI-Based Parallel Soft Clustering

Member	Role
Khansa Danish	Core Algorithm & Parallelisation (Member 1)
Ali Hamza	Clinical Text Processing & Feature Engineering (Member 2)
Zain Khan	Metrics, Testing & Validation (Member 3)
Arham Jumshaid	Data Distribution, Load Balancing & Visualisation (Member 4)

Course: Parallel and Distributed Computing
Submission: Milestone 2
Dataset: MTSamples Clinical Notes (4,943 documents, 500 TF-IDF features)
Date: April 2026

This report documents Milestone 2: MPI parallelisation of the FCM algorithm
built on top of the serial baseline delivered in Milestone 1.

Contents

1	Project Overview and Milestone 2 Scope	2
1.1	Milestone 2 Objectives (Member 1)	2
2	Dataset: Member 2's Clinical Feature Vectors	2
2.1	Data Characteristics	2
2.2	Medical Specialties in the Dataset	2
2.3	Domain Label Mapping for Domain-Guided Initialisation	3
3	Member 1 — Parallel Algorithm Design (Khansa Danish)	3
3.1	Overview of Parallelisation Strategy	3
3.2	Data Distribution: Block Decomposition	3
3.3	Parallel E-Step: Membership Computation	4
3.4	Parallel M-Step: Centroid Update with MPI_Allreduce	4
3.5	Parallel Convergence Detection	5
3.6	MPI Communication Summary Per Iteration	5
3.7	Parallel Initialisation Strategies	6
3.8	Main Parallel Loop Structure	6
3.9	Output Collection	7
4	File Structure	8
5	Setup and Execution Guide	8
5.1	Environment (WSL2 on Windows)	8
5.2	Project Setup	8
5.3	Building	8
5.4	Running	9
5.5	Expected Output	9
5.6	Note on <code>-oversubscribe</code>	9
6	Connection to Other Members' Work	10
6.1	From Member 2 (Ali Hamza)	10
6.2	To Member 3 (Zain Khan)	10
6.3	To Member 4 (Arham Jumshaid)	10
7	Other Members' Sections	10
	Member 2 — Clinical Text Processing & Feature Engineering	10
	Member 3 — Metrics, Testing & Validation	10
	Member 4 — Data Distribution, Load Balancing & Visualisation	11
8	Milestone 2 Deliverable Checklist	11
9	Conclusion	11

1. Project Overview and Milestone 2 Scope

Milestone 2 parallelises the Fuzzy C-Means (FCM) algorithm developed in Milestone 1 using OpenMPI. The dataset used is the real clinical notes corpus delivered by Member 2 (Ali Hamza): **4,943 MTSamples clinical transcriptions** represented as **500-dimensional TF-IDF feature vectors** across **40 medical specialties**. The parallel implementation targets 10 clusters ($C = 10$), chosen to capture the dominant specialty groupings present in the data.

1.1. Milestone 2 Objectives (Member 1)

- Distribute the 4943×500 feature matrix across MPI ranks using block decomposition and `MPI_Scatterv`.
- Parallelise the E-step (membership computation) — embarrassingly parallel, no communication.
- Parallelise the M-step (centroid update) using `MPI_Allreduce` to aggregate partial sums.
- Implement distributed convergence detection via `MPI_Allreduce` on local Frobenius norm contributions.
- Adapt all three initialisation strategies (Random, K-Means++, Domain-guided) for MPI execution.
- Instrument each iteration with `MPI_Wtime` for Member 3’s scaling analysis.

2. Dataset: Member 2’s Clinical Feature Vectors

2.1. Data Characteristics

Member 2 (Ali Hamza) delivered the following files from his NLP preprocessing pipeline:

File	Contents	Size
<code>features.csv</code>	TF-IDF feature matrix, one row per document	4943×500
<code>feature_names.csv</code>	Names of the 500 TF-IDF features (medical terms)	500 rows
<code>specialty_labels.csv</code>	Medical specialty string per document	4943 rows

Table 1: Files delivered by Member 2 for Milestone 2

2.2. Medical Specialties in the Dataset

The dataset spans 40 distinct medical specialties. The five most frequent are:

Specialty	Documents	% of corpus
Surgery	1085	21.9%
Consult - History & Phy.	515	10.4%
Cardiovascular / Pulmonary	369	7.5%
Orthopedic	351	7.1%
Radiology	271	5.5%
Other (35 specialties)	2352	47.6%
Total	4943	100%

Table 2: Top medical specialties in the MTSamples corpus

2.3. Domain Label Mapping for Domain-Guided Initialisation

The 40 specialty strings are mapped to integer cluster indices $\{0, \dots, C-1\}$ using modular arithmetic ($\text{label_id} \bmod C$). This allows the domain-guided initialisation strategy to seed cluster centroids from clinically meaningful groupings even when $C < 40$.

3. Member 1 — Parallel Algorithm Design (Khansa Danish)

3.1. Overview of Parallelisation Strategy

The FCM algorithm is parallelised across P MPI processes using **block data decomposition**. The $N = 4943$ document feature vectors are divided into P contiguous blocks, one per rank. The key insight is that the E-step is *embarrassingly parallel* while the M-step requires a single global reduction.

3.2. Data Distribution: Block Decomposition

At startup, rank 0 reads the full $N \times F$ feature matrix and distributes rows to all ranks using `MPI_Scatterv`. Variable counts are used because N may not divide evenly by P :

$$\begin{aligned} \text{local_start}(r) &= r \cdot \left\lfloor \frac{N}{P} \right\rfloor + \min(r, N \bmod P) \\ \text{local_n}(r) &= \left\lfloor \frac{N}{P} \right\rfloor + \mathbf{1}[r < N \bmod P] \end{aligned}$$

For example, with $N = 4943$ and $P = 4$:

Rank	Start Row	Local Rows	Data (doubles)
0	0	1236	618,000
1	1236	1236	618,000
2	2472	1236	618,000
3	3708	1235	617,500
Total		4943	2,471,500

Table 3: Block decomposition of 4943 documents across 4 MPI ranks

3.3. Parallel E-Step: Membership Computation

The E-step updates u_{ij} for each document i and cluster j :

$$u_{ij} = \frac{1}{\sum_{k=1}^C \left(\frac{\|x_i - c_j\|_2}{\|x_i - c_k\|_2} \right)^{\frac{2}{m-1}}}$$

Each rank computes this independently for its local rows only. **No MPI communication is required during the E-step** — it is embarrassingly parallel. Each rank reads the globally shared centroid array (which all ranks maintain identically after each M-step) and writes to its local membership slice.

```

1 void fcm_mpi_update_membership(FCMMpiModel *m) {
2     int ln = m->local_n, F = m->F, C = m->C;
3     double exp = 2.0 / (FCM_M - 1.0);
4     double *dist = malloc(C * sizeof(double));
5
6     for (int i = 0; i < ln; i++) {           /* local rows only */
7         for (int j = 0; j < C; j++)
8             dist[j] = l2_distance(m->local_data + i*F,
9                                   m->centroids + j*F, F);
10        for (int j = 0; j < C; j++) {
11            double sum = 0.0;
12            for (int k = 0; k < C; k++)
13                sum += pow(dist[j] / dist[k], exp);
14            m->local_U[i*C + j] = 1.0 / sum;
15        }
16    }
17    free(dist);
18    /* No MPI calls needed -- embarrassingly parallel */
19 }

```

Listing 1: Parallel E-step — each rank processes only its local rows

3.4. Parallel M-Step: Centroid Update with MPI_Allreduce

The M-step requires the weighted sum of *all* N documents to compute each centroid:

$$c_j = \frac{\sum_{i=1}^N u_{ij}^m \cdot x_i}{\sum_{i=1}^N u_{ij}^m}$$

Each rank computes partial sums over its local rows. `MPI_Allreduce` with `MPI_SUM` aggregates these partial sums globally. Crucially, `MPI_Allreduce` distributes the result back to *all* ranks simultaneously — no separate broadcast is needed.

```

1 void fcm_mpi_update_centroids(FCMMpiModel *m) {
2     /* Step 1: each rank computes local partial sums */
3     for (int i = 0; i < m->local_n; i++) {
4         for (int j = 0; j < C; j++) {
5             double u_m = pow(m->local_U[i*C + j], FCM_M);
6             local_den[j] += u_m;

```

```

7         for (int f = 0; f < F; f++)
8             local_num[j*F + f] += u_m * m->local_data[i*F + f
9                 ];
10    }
11    /* Step 2: global reduction -- 2 Allreduce calls per M-step */
12    MPI_Allreduce(local_num, global_num, C*F,
13                  MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
14    MPI_Allreduce(local_den, global_den, C,
15                  MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
16    /* Step 3: all ranks compute identical centroids */
17    for (int j = 0; j < C; j++)
18        for (int f = 0; f < F; f++)
19            m->centroids[j*F+f] = global_num[j*F+f] / global_den[
20                j];

```

Listing 2: Parallel M-step — local partial sums aggregated via MPI_Allreduce

3.5. Parallel Convergence Detection

Each rank computes its local contribution to the global Frobenius norm squared. A single MPI_Allreduce sums these, and the square root is taken once. This avoids any master-slave bottleneck:

$$\|U^{\text{new}} - U^{\text{old}}\|_F^2 = \sum_{r=0}^{P-1} \underbrace{\sum_{i \in \text{rank}_r} \sum_{j=1}^C (u_{ij}^{\text{new}} - u_{ij}^{\text{old}})^2}_{\text{local_norm}_r^2}$$

```

1 double fcm_mpi_convergence(FCMMpiModel *m) {
2     double local_norm2 = 0.0;
3     for (int i = 0; i < m->local_n; i++)
4         for (int j = 0; j < m->C; j++) {
5             double diff = m->local_U[i*m->C+j]
6                 - m->local_U_old[i*m->C+j];
7             local_norm2 += diff * diff;
8         }
9     double global_norm2 = 0.0;
10    MPI_Allreduce(&local_norm2, &global_norm2, 1,
11                  MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
12    return sqrt(global_norm2); /* same on all ranks */
13 }

```

Listing 3: Distributed convergence detection — no master bottleneck

3.6. MPI Communication Summary Per Iteration

Each FCM iteration requires exactly **3 MPI collective operations**:

Step	MPI Call	Purpose	Data Volume
M-step (numerator)	MPI_Allreduce	Sum partial weighted vectors	$C \times F \times 8$ bytes
M-step (denominator)	MPI_Allreduce	Sum partial membership weights	$C \times 8$ bytes
Convergence	MPI_Allreduce	Sum local Frobenius norm ²	1×8 bytes
Total/iter	3 Allreduce		≈ 40 KB ($C=10, F=500$)

Table 4: MPI communication per FCM iteration ($C = 10, F = 500$)

The communication volume is **fixed at 40 KB per iteration regardless of N** — it depends only on $C \times F$, not the dataset size. This is why the algorithm scales well: adding more ranks reduces local computation while communication overhead stays constant.

3.7. Parallel Initialisation Strategies

All three initialisation strategies from Milestone 1 are adapted for MPI:

Strategy	MPI Adaptation	Key Calls
Random	Rank 0 generates full U matrix, MPI_Scatterv distributes row slices to each rank	MPI_Scatterv
K-Means++	MPI_Gatherv brings all data to rank 0 for seeding (sampled). Centroids broadcast to all. Each rank initialises its local U slice from nearest centroid	MPI_Gatherv, MPI_Bcast
Domain-Guided	Rank 0 computes class-mean centroids using gathered data and specialty labels. Labels scattered to ranks for local U initialisation	MPI_Gatherv, MPI_Bcast, MPI_Scatterv

Table 5: MPI adaptation of each initialisation strategy

3.8. Main Parallel Loop Structure

```

1 void fcm_mpi_run(FCMMpiModel *m, InitStrategy strategy,
2                 int *domain_labels) {
3     /* 1. Initialise U (strategy-dependent, see above) */
4     fcm_mpi_init_(m, ...);

```

```

5
6  /* 2. Initial centroids from U */
7  fcm_mpi_update_centroids(m);    /* M-step + 2x Allreduce */
8
9  for (int iter = 1; iter <= FCM_MAX_ITER; iter++) {
10     double t0 = MPI_Wtime();
11
12     copy_flat(m->local_U_old, m->local_U, local_n * C);
13
14     fcm_mpi_update_membership(m); /* E-step: no comm */
15     fcm_mpi_update_centroids(m); /* M-step: 2 Allreduce */
16     double delta = fcm_mpi_convergence(m); /* 1 Allreduce */
17
18     m->iter_times[iter-1] = MPI_Wtime() - t0;
19
20     if (delta < FCM_EPSILON) break;
21 }
22 }

```

Listing 4: Main parallel FCM loop with timing instrumentation

3.9. Output Collection

After convergence, `MPI_Gatherv` collects all local U slices to rank 0, which writes the full membership matrix and centroid files for Member 3:

File	Dimensions	Use
membership_mpi_random.csv	4943×10	Silhouette, DB-index (Member 3)
membership_mpi_kmeanspp.csv	4943×10	Strategy comparison
membership_mpi_domain.csv	4943×10	Strategy comparison
centroids_mpi_random.csv	10×500	Top medical terms per cluster
centroids_mpi_kmeanspp.csv	10×500	Cluster interpretation
centroids_mpi_domain.csv	10×500	Cluster interpretation

Table 6: Output files produced by the parallel FCM implementation

4. File Structure

File	Purpose	Owner
fcm_mpi.h	All structs, constants, function prototypes	Member 1
fcm_mpi.c	Full parallel FCM: init, E-step, M-step, convergence, I/O	Member 1
main_mpi.c	Entry point: argument parsing, timing, output	Member 1
Makefile_mpi	Build and run rules	Member 1
features.csv	TF-IDF feature matrix	Member 2
specialty_labels.csv	Medical specialty labels	Member 2

Table 7: Milestone 2 file structure

5. Setup and Execution Guide

5.1. Environment (WSL2 on Windows)

All code runs inside WSL2 (Ubuntu). OpenMPI must be installed:

```
sudo apt update
sudo apt install -y openmpi-bin openmpi-common libopenmpi-dev
mpicc --version      # verify: should show GCC version
mpirun --version     # verify: should show Open MPI 4.x
```

Listing 5: Install OpenMPI in WSL2

5.2. Project Setup

```
# Create project folder inside Ubuntu (NOT from Windows Explorer)
mkdir ~/fcm_milestone2
cd ~/fcm_milestone2

# Copy all 4 source files here:
#   fcm_mpi.h  fcm_mpi.c  main_mpi.c  Makefile_mpi
# Copy Member 2's data files here:
#   features.csv  specialty_labels.csv

# Rename Makefile
mv Makefile_mpi Makefile
```

Listing 6: Setting up the project directory

5.3. Building

```
make
# Expected output: "Build successful --> ./fcm_mpi"
```

Listing 7: Build the parallel FCM executable

5.4. Running

```
# K-Means++ with 4 processes (recommended)
mpirun --oversubscribe -np 4 ./fcm_mpi features.csv
      specialty_labels.csv 1

# Random initialisation with 4 processes
mpirun --oversubscribe -np 4 ./fcm_mpi features.csv
      specialty_labels.csv 0

# Domain-guided with 4 processes (uses specialty labels)
mpirun --oversubscribe -np 4 ./fcm_mpi features.csv
      specialty_labels.csv 2

# Or use Makefile shortcuts:
make run          # K-Means++, 4 processes
make run_random   # Random,      4 processes
make run_domain   # Domain,      4 processes
make run_np NP=8   # K-Means++, 8 processes
```

Listing 8: Running with different process counts and strategies

5.5. Expected Output

```
[rank 0] Loaded 4943 x 500 feature matrix from 'features.csv'
[init] K-Means++ init complete (seeds chosen).
[fcm_mpi] N=4943 F=500 C=10 P=4 m=2.0 eps=1e-05
[fcm_mpi] iter    1  delta=X.XXXXXXXX  time=X.XXXXs
[fcm_mpi] iter    2  delta=X.XXXXXXXX  time=X.XXXXs
...
[fcm_mpi] Converged at iter N (delta=X.XXe-06)
[fcm_mpi] -- Summary --
[fcm_mpi] Cluster distribution:
  Cluster  0 : XXXX documents (XX.X%)
...
[fcm_mpi] Total wall time : X.XXXX s
[fcm_mpi] Avg time/iter   : X.XXXX s
[fcm_mpi] Membership saved --> membership_mpi_kmeanspp.csv
[fcm_mpi] Centroids saved  --> centroids_mpi_kmeanspp.csv
```

Listing 9: Expected terminal output when running

5.6. Note on `--oversubscribe`

On a laptop with fewer physical cores than requested processes, MPI will complain about insufficient “slots”. The `--oversubscribe` flag tells MPI to ignore this limit and run multiple processes on the same core. This is expected on a single-machine WSL2 setup and does not affect correctness. On the IBA HPC cluster, multiple real nodes are available and this flag is not needed.

6. Connection to Other Members' Work

6.1. From Member 2 (Ali Hamza)

Member 2's `features.csv` and `specialty_labels.csv` are directly consumed by `fcm_mpi_load_and_scale` and `fcm_mpi_load_labels()`. No preprocessing is needed — the TF-IDF values are already in $[0, 1]$ as required by FCM.

6.2. To Member 3 (Zain Khan)

After running, Member 3 receives:

- Six CSV files (membership + centroids for each strategy).
- Per-iteration wall times in `model->iter_times[]` — available for speedup analysis.
- Run with varying `-np` values (1, 2, 4, 8) to collect strong scaling data.

6.3. To Member 4 (Arham Jumshaid)

The 4943×10 membership matrices are ready for heatmap visualisation. Each row sums to 1 and represents one clinical document's soft cluster memberships across 10 groups.

7. Other Members' Sections

Member 2 — Clinical Text Processing & Feature Engineering (Ali Hamza)

NLP Preprocessing Pipeline

Medical Entity Extraction and Negation Detection

TF-IDF Vectorisation

Feature Matrix Delivery

Member 3 — Metrics, Testing & Validation (Zain Khan)

Clustering Quality Metrics (Silhouette, Davies-Bouldin)

Strong Scaling Results

Weak Scaling Results

MPI Communication Overhead Analysis

Member 4 — Data Distribution, Load Balancing & Visualisation (Arham Jumshaid)

Block vs Cyclic Distribution Comparison

Dynamic Load Balancing

Membership Heatmap Visualisation

Top Medical Terms Per Cluster

8. Milestone 2 Deliverable Checklist

Task	Status	Location
Block decomposition of N docs across P ranks	Complete	<code>fcm_mpi.c</code>
<code>MPI_Scatterv</code> data distribution	Complete	<code>fcm_mpi_load_and_scatter()</code>
Parallel E-step (no communication)	Complete	<code>fcm_mpi_update_membership()</code>
Parallel M-step with <code>MPI_Allreduce</code>	Complete	<code>fcm_mpi_update_centroids()</code>
Distributed convergence via <code>MPI_Allreduce</code>	Complete	<code>fcm_mpi_convergence()</code>
Random initialisation (MPI-adapted)	Complete	<code>fcm_mpi_init_random()</code>
K-Means++ initialisation (MPI-adapted)	Complete	<code>fcm_mpi_init_kmeanspp()</code>
Domain-guided init using specialty labels	Complete	<code>fcm_mpi_init_domain()</code>
Per-iteration timing with <code>MPI_Wtime</code>	Complete	<code>fcm_mpi_run()</code>
<code>MPI_Gatherv</code> output collection	Complete	<code>fcm_mpi_gather_and_save()</code>
CSV output for Member 3 (6 files)	Complete	<code>main_mpi.c</code>
Tested on MTSamples data (4943 docs, 500 feat.)	Complete	Verified compile + load

Table 8: Member 1 Milestone 2 deliverable checklist

9. Conclusion

Milestone 2 delivers a fully functional MPI-parallel Fuzzy C-Means implementation operating on Member 2's real MTSamples clinical dataset of 4,943 documents with 500 TF-IDF features. The key design decisions are:

- **E-step is embarrassingly parallel** — zero communication per document.
- **M-step uses exactly 2 `MPI_Allreduce` calls** — communication volume is fixed at ≈ 40 KB per iteration regardless of dataset size.
- **Convergence uses 1 `MPI_Allreduce`** — no master-slave bottleneck.
- **Total: 3 collective operations per iteration** — minimum possible for correct parallel FCM.

- Communication volume is $O(C \cdot F)$, not $O(N)$ — the algorithm scales efficiently as N grows.
- All three initialisation strategies are MPI-adapted, with K-Means++ using subsampled seeding for practical speed on high-dimensional data.

The implementation is ready for Member 3's strong and weak scaling experiments by simply varying the `-np` argument to `mpirun`.